Monolithic Architecture in OS

Last Updated: 21 Aug 2025

The monolithic operating system is a very simple operating system where the kernel directly controls device management, memory management, file management, and process management. All of the system's resources are accessible to the kernel. Every part of the operating system is contained within the kernel in monolithic systems. Monolithic architecture-based operating systems were initially introduced in the 1970s.

The monolithic kernel is another name for the monolithic operating system. This is an outdated operating system that banks employ for menial jobs like time-sharing and batch processing. All hardware components are managed by the monolithic kernel, which functions as a virtual machine.

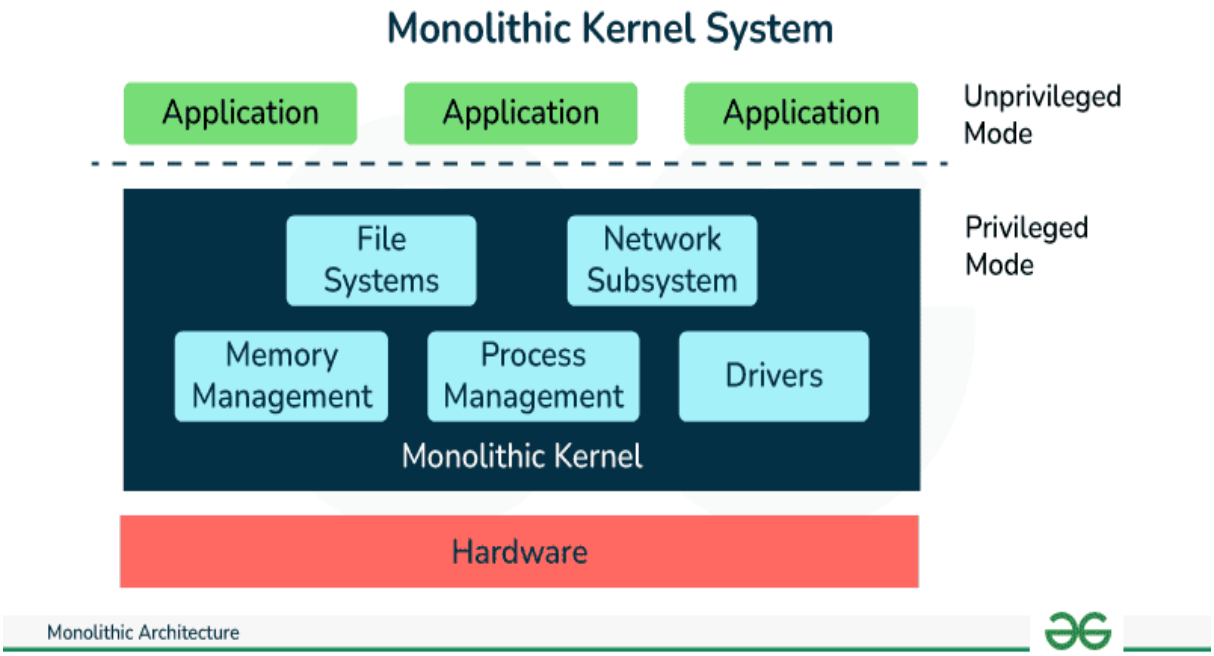
Unlike a microkernel, which has fewer tasks, it is not the same. The two sections of a microkernel are called kernel space and user space. IPC (inter-process communication) is the means by which both components speak with one another. One benefit of a microkernel is that in the event of a failure, the other server assumes control.

What is Monolithic Architecture?

In a monolithic architecture, the operating system kernel is designed to provide all operating system services, including memory management, process scheduling, device drivers, and file systems, in a single, large binary. This means that all code runs in kernel space, with no separation between kernel and user-level processes.

The main advantage of a monolithic architecture is that it can provide high performance since system calls can be made directly to the kernel without the overhead of message passing between user-level processes. Additionally, the design is simpler, since all operating system services are provided by a single binary.

However, there are also some disadvantages to monolithic architecture. One major disadvantage is that it can lead to a less secure and less stable operating system. Since all code runs in kernel space, any vulnerabilities or bugs in the kernel can potentially affect the entire system. Additionally, if a user-level process crashes, it can bring down the entire system, since there is no separation between kernel and user-level processes.



Monolithic Architecture

Two basic instances of monolithic operating systems are DOS and CP/M. Operating systems that share a single address space with applications are DOS and CP/M.

- System variables and the application area are the first two addresses in the 16-bit address space of CP/M. The operating system is concluded with the following three components: BIOS (Basic Input/Output System), BDOS (Basic Disk Operating System), and CCP (Console Command Processor).
- The array of interrupt vectors and system variables are at the beginning of DOS's 20-bit address space. Next comes the

application area and resident portion of DOS, and lastly a memory block utilized by the BIOS and visual card.

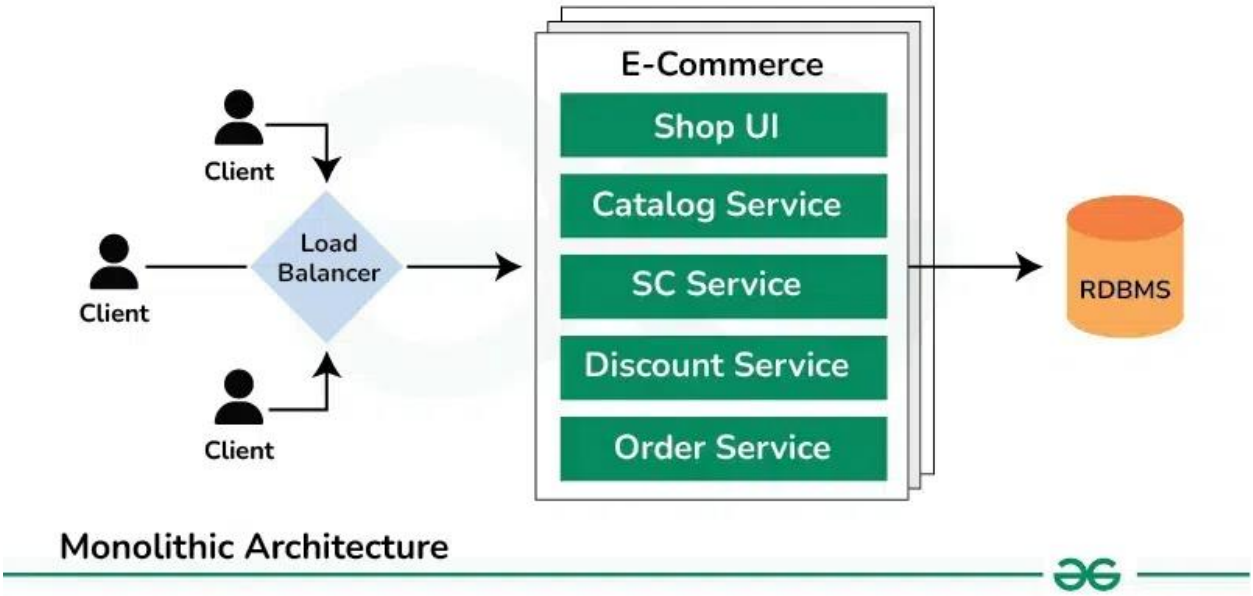
Characteristics of Monolithic Architecture

- **Single Executable:** The entire application is packaged and deployed as a single executable file. All components and modules are bundled together.
- **Tight Coupling:** The components and modules within the application are highly interconnected and dependent on each other. Changes made to one component may require modifications in other parts of the application.
- **Shared Memory:** All components within the application share the same memory space. They can directly access and modify shared data structures.
- **Monolithic Deployment:** The entire application is deployed as a single unit. Updates or changes to the application require redeploying the entire monolith.
- **Centralized Control Flow:** The control flow within the application is typically managed by a central module or a main function. The flow of execution moves sequentially from one component to another.

Example of Monolithic Architecture

Before Learning Microservices, we always know why we do not use Monolithic architecture nowadays. So that it will help us to understand the Microservices more clearly. Monolithic Architecture is like a big container, wherein all the software components of an app are assembled and tightly coupled, i.e., each component fully depends on each other.

Example: Here's an example of an e-commerce app in a monolithic architecture:



Monolithic Architecture

- **Shop UI (Frontend)**: The **Shop UI** is the main user interface where users browse products, add them to the cart, apply discounts, and place orders.
- **Catalog Service**: The **Catalog Service** handles all product-related information, such as fetching the product list, product details, availability, and pricing.
- **Shopping Cart (SC) Service:** It manages the user's cart operations, such as adding products, removing products, and calculating the total.
- **Discount Service**: The **Discount Service** handles any discount logic, such as applying promotional codes or special offers.
- **Order Service**: The **Order Service** processes user orders, including handling payment, order creation, and communication with shipping providers.

How the Monolithic Architecture Works Together?

- All these services are part of a **single codebase** and deployed as one application. They communicate via **direct function calls**, and any changes to one service (e.g., the Catalog Service) require redeployment of the entire application.

- A **single database** is shared by all services, meaning the Catalog Service, Shopping Cart Service, Discount Service, and Order Service all access the same database tables.
- The entire application is built, tested, and deployed as a single unit, running on a single server or instance (or scaled horizontally by replicating the entire monolith).

Advantages of Monolithic Architecture

Below are some advantages of Monolithic Architecture.

- **High performance:** Monolithic kernels can provide high performance since system calls can be made directly to the kernel without the overhead of message passing between user-level processes.
- **Simplicity:** The design of a monolithic kernel is simpler since all operating system services are provided by a single binary. This makes it easier to develop, test and maintain.
- **Broad hardware support:** Monolithic kernels have broad hardware support, which means that they can run on a wide range of hardware platforms.

- **Low overhead:** The monolithic kernel has low overhead, which means that it does not require a lot of system resources, making it ideal for resource-constrained devices.
- **Easy access to hardware resources:** Since all code runs in kernel space, it is easy to access hardware resources such as network interfaces, graphics cards, and sound cards.
- **Fast system calls:** Monolithic kernels provide fast system calls since there is no overhead of message passing between user-level processes.
- **Good for general-purpose operating systems:** Monolithic kernels are good for general-purpose operating systems that require a high degree of performance and low overhead.
- **Easy to develop drivers:** Developing device drivers for monolithic kernels is easier since they are integrated into the kernel.

Overall, a monolithic architecture provides high performance, simplicity, and broad hardware support. It is ideal for general-purpose operating systems that require a high degree of performance and low overhead. However, it may come with some trade-offs in terms of security, stability, and flexibility.

Disadvantages of Monolithic Architecture

Below are some disadvantages of Architecture.

- **Large and Complex Applications:** For large and complex applications in monolithic, it is difficult for maintenance because they are dependent on each other.
- **Slow Development:** It is because, for modify an application we have to redeploy whole application instead of updates part. It takes more time or slow development.
- **Unscalable:** Each copy of the application will access the whole data which make more memory consumption. We cannot scale each component independently.
- **Unreliable:** If one services goes down, then it affects all the services provided by the application. It is because all services of applications are connected to each other.
- **Inflexible:** Really difficult to adopt new technology. It is because we have to change the whole application technology.

Comment

N

niranjan shukla